UNIVERSIDADE FEDERAL DO ABC - UFABC
Centro de Engenharia, Modelagem e Ciencias Sociais Aplicadas (CECS)
Disciplina: MCTA028-15 - Programacao Estruturada | Quadrimestre 2026.3
Docentes: Prof. Roberto Sadao Yokoyama e Prof. Luneque Del Rio de Souza e Silva Jr.

RELATORIO TECNICO DE IMPLEMENTACAO

GarchY: Compactador e Descompactador Huffman em ANSI C

INTEGRANTES DA EQUIPE E REPOSITORIO:
- Integrante 1 (jrfeltran): Min-Heap, Arvore de Huffman e Geracao de Codigos
- Integrante 2 (RAFAELMX30): BitStream, Cabecalho Compacto e Rotina de Compressao
- Integrante 3 (CWAF-Renan): Rotina de Descompressao e Integridade CRC-32
- Integrante 4 (Rodrigo Araujo - Martter): Menu, Tratamento de Caminhos, Metricas e Relatorio

1. REQUISITOS IMPLEMENTADOS E FUNCIONAIS

O projeto GarchY foi desenvolvido em conformidade com as diretrizes do Trabalho de Implementacao de Programacao Estruturada (2026.3). O objetivo central foi implementar uma ferramenta completa de compressao e descompressao de arquivos sem perdas (lossless) em padrao formal ANSI C, operando tanto sobre arquivos textuais quanto binarios.

Dentre os requisitos funcionais integralmente cumpridos, destacam-se:

1. Interface Interativa no Terminal: Menu estruturado permitindo ao usuario escolher entre compressao, descompressao, inspeção da tabela de codigos de Huffman, teste automatizado de integridade (roundtrip) e informacoes sobre o sistema.

2. Tratamento Robusto de Caminhos: Aceitacao de caminhos relativos e absolutos, sanitizacao de quebras de linha residuais (\r e \n), remocao automatica de aspas (facilitando arrastar arquivos no terminal) e validacao de existencia previa dos arquivos.

3. Extensao Personalizada: O compressor gera por padrao arquivos com extensao '.abcz'. Na descompressao, o programa sugere o nome original expurgando a extensao e permite ao usuario especificar qualquer destino desejado.

4. Preservacao dos Originais e Corretude: Os arquivos originais sao estritamente preservados em modo somente-leitura. Em caso de qualquer erro de I/O ou alocacao, arquivos parciais corrompidos sao automaticamente eliminados pelo sistema.

5. Metricas de Eficiencia Espacial e Temporal: Exibicao instantanea dos tamanhos em Bytes (e formatado em KB/MB), Razao de Compressao (R = Sc / So), porcentagem de reducao de espaco, tempo cronometrado via clock() e velocidade de processamento.

2. FUNDAMENTACAO CONCEITUAL DOS ALGORITMOS

2.1 Teoria da Informacao e Entropia de Shannon

A compressao estatistica sem perdas encontra seu limite teorico na Entropia de Shannon H(X). Em um arquivo digital, cada byte representa um simbolo dentre 256 valores possiveis (0 a 255). Se as frequencias forem desiguais, atribuir um comprimento fixo de 8 bits por byte gera redundancia informacional. O algoritmo de David Huffman (1952) resolve esse problema ao gerar codigos binarios de tamanho variavel que satisfazem a propriedade do prefixo (nenhum codigo e prefixo de outro).

2.2 Fila de Prioridade (Min-Heap) e Construcao da Arvore

A construcao da arvore de Huffman e realizada de baixo para cima (bottom-up) por meio de uma fila de prioridade baseada em Min-Heap com complexidade O(N log N). Inicialmente, cada byte que aparece no arquivo e encapsulado em um no folha. A cada iteracao, os dois nos com as menores frequencias sao retirados do heap e combinados em um novo no intermediario, cuja frequencia e a soma dos filhos. Esse no e reinserido no heap ate que reste apenas a raiz.

Para assegurar que a arvore reconstruida na descompressao seja 100% identica a arvore da compressao sem a necessidade de gravar ponteiros no disco, implementou-se um criterio deterministico de desempate: nos com a mesma frequencia sao ordenados pelo identificador sequencial de criacao ('ordem').

2.3 Verificacao de Integridade por Checksum CRC-32

Para a verificacao de integridade (funcionalidade extra), adotou-se o algoritmo Cyclic Redundancy Check de 32 bits (CRC-32) padrao IEEE 802.3 com o polinomio 0xEDB88320 em representacao invertida. O CRC-32 processa os dados incrementalmente em tempo linear O(N) utilizando uma tabela de dispersao de 256 entradas pre-calculada. O valor resultante e gravado no cabecalho e recalculado na descompressao, garantindo a deteccao de qualquer corrupcao de bit ou truncamento.

3. DECISOES DE PROJETO, ESTRUTURAS DE DADOS E CODIGO

O software foi integralmente projetado segundo os principios de modularizacao e encapsulamento de Tipos Abstratos de Dados (TADs), distribuido em arquivos de cabecalho (.h em include/) e implementacoes (.c em src/).

3.1 TAD Min-Heap e Arvore de Huffman (Integrante 1)

O Min-Heap foi implementado sobre um vetor de ponteiros com capacidade fixa de 256 elementos (quantidade maxima de simbolos). O gerenciamento de memoria e rigoroso: cada malloc e acompanhado de verificacao de ponteiro nulo e a arvore e totalmente desalocada em pos-ordem com huff_liberar_arvore().

```
/* Trecho representativo: criterio deterministico de desempate no Min-Heap */
static int compara(const HuffNode *a, const HuffNode *b) {
    if (a->freq < b->freq) return -1;
    if (a->freq > b->freq) return 1;
    return a->ordem - b->ordem; /* Desempate pela ordem de insercao */
}
```

3.2 BitStream e Serializacao Compacta do Cabecalho (Integrante 2)

Como os codigos de Huffman possuem comprimentos fracionarios de byte, implementou-se o BitWriter e BitReader, que acumulam bits em buffers de 1 byte enviados ao disco quando cheios (MSB primeiro). O cabecalho .abcz armazena a assinatura 'ABCZ', a versao do formato, o tamanho original (8 bytes) e o checksum (4 bytes). Em seguida, armazena a tabela de frequencias usando codificacao varint (7 bits por byte com bit mais significativo indicando continuacao), garantindo que arquivos pequenos nao sofram sobrecarga desnecessaria de cabecalho.

```
/* Trecho representativo: BitWriter acumulando bits MSB primeiro */
int bw_put_bit(BitWriter *bw, int bit) {
    bw->buf = (unsigned char)((bw->buf << 1) | (bit ? 1 : 0));
    bw->n_bits++;
    if (bw->n_bits == 8) {
        if (fputc(bw->buf, bw->f) == EOF) return -1;
        bw->buf = 0; bw->n_bits = 0;
    }
    return 0;
}
```

3.3 Rotinas de Compressao e Descompressao em 2 Passadas (Integrantes 2 e 3)

Para nao comprometer a memoria RAM em arquivos extensos, a compressao opera em blocos de 64 KB (TAM_BLOCO) e faz duas passadas sobre o arquivo original: na 1a passada le os blocos, conta frequencias e calcula o CRC-32; em seguida constroi a arvore e os codigos; na 2a passada substitui cada byte pela sua sequencia de bits correspondente.

Na descompressao, os bits sao consumidos um a um para navegar na arvore reconstruida (bit 0 = esquerda, bit 1 = direita). Ao alcancar um no folha, o byte correspondente e emitido e acumulado no bloco de saida, parando precisamente quando o tamanho original registrado no cabecalho e alcancado, descartando quaisquer bits de preenchimento (padding) do ultimo byte.

3.4 Menu, Tratamento de Caminhos e Metricas (Integrante 4)

O modulo principal (src/main.c) fornece a camada de interacao com o usuario. A funcao normalizar_caminho() remove espacos e aspas duplas adicionadas automaticamente pelo terminal ao arrastar arquivos. A contagem de tempo emprega clock() para medicao com resolucao de milissegundos e as grandezas sao convertidas automaticamente em B, KB e MB.

4. CASOS DE TESTE E MENSURACAO DE DESEMPENHO

Conforme requisitado no edital, o programa foi testado sob tres cenarios de redundancia estatistica com arquivos padronizados de 500 KB (512.000 Bytes), comparando os resultados do GarchY contra o compressor comercial gzip (modo Deflate):

- Cenario 1 (Alta Redundancia): Padrao repetitivo homogeneo com reduzido conjunto de caracteres (ex: sequencias biologicas de DNA).
- Cenario 2 (Media Redundancia): Texto corrido estruturado em linguagem natural (literatura e documentacao tecnica).
- Cenario 3 (Baixa Redundancia): Dados pseudo-aleatorios com distribuicao uniforme de bytes (alta entropia, ~8 bits/simbolo).

Tabela 1: Comparativo de Eficiencia Espacial e Integridade (500 KB / 512.000 Bytes)

Cenario	Tam. Orig.	Huffman (Sc)	Razao (R)	Economia	Gzip Tam.	CRC-32
Alta Redundancia	512.000 B	231.263 B	0,4517	54,83%	1.578 B	OK 100%
Media Redundancia	512.000 B	283.361 B	0,5534	44,66%	5.604 B	OK 100%
Baixa Redundancia	512.000 B	512.787 B	1,0015	-0,15%	512.196 B	OK 100%

Tabela 2: Eficiencia Temporal de Compressao e Descompressao

Cenario	Comp. Huffman	Comp. Gzip	Decomp. Huffman	Decomp. Gzip
Alta Redundancia	51,10 ms	12,06 ms	48,65 ms	23,47 ms
Media Redundancia	59,73 ms	17,25 ms	49,37 ms	121,85 ms
Baixa Redundancia	36,83 ms	144,73 ms	56,70 ms	161,16 ms

Analise Critica e Comparativa com a Ferramenta Comercial (Gzip):

1. Desempenho do Huffman: O GarchY obteve excelente reducao de volume nos dados textuais e repetitivos, reduzindo 54,83% no cenario de alta redundancia e 44,66% em linguagem natural com tempos sempre inferiores a 60 milissegundos. Este resultado comprova que o algoritmo atingiu a taxa otima esperada para modelos estaticos de Huffman por byte.

2. Diferenca Estrutural para o Gzip: O gzip utiliza o algoritmo Deflate, que combina codificacao de Huffman com LZ77 (janela deslizante por dicionario). Em sequencias de repeticao longa (como frases repetidas no cenario de alta redundancia), o LZ77 substitui toda a repeticao por ponteiros curtos de (distancia, tamanho), gerando taxas de compressao drasticamente menores (R = 0,0031). Por outro lado, o Huffman puro avalia apenas a frequencia individual de cada byte, sem explorar correlacao espacial entre caracteres adjacentes.

3. Comportamento em Baixa Redundancia (Alta Entropia): Em dados pseudo-aleatorios com distribuicao uniforme, a entropia por byte aproxima-se de 8 bits/simbolo. Conforme estabelecido pela Teoria da Informacao, nenhum algoritmo sem perdas consegue compactar dados aleatorios. Ambos os programas registraram expansao residual insignificante devida aos metadados de cabecalho (+0,15% no GarchY e +0,04% no Gzip).

4. Corretude e Integridade: Em todos os testes, os arquivos recuperados foram confirmados como 100%

identicos aos originais tanto pela comparacao de hash SHA-256 quanto pelo CRC-32 integrado.

5. PARTICIPACAO E ENTREGAS INDIVIDUAIS DOS INTEGRANTES

O trabalho foi dividido em 4 frentes complementares e cooperativas, conforme registrado no repositrio oficial do projeto:

- Integrante 1 (jrfeltran): Implementacao da fila de prioridade Min-Heap (heap.c/heap.h), construcao da arvore binaria de Huffman com desempate deterministico e algoritmo de geracao dos codigos binarios de tamanho variavel (huffman.c/huffman.h).
- Integrante 2 (RAFAELMX30): Implementacao do leitor e escritor serial de bits BitStream (bitio.c/bitio.h), formato de cabecalho compacto com codificacao varint (cabecalho.c/cabecalho.h) e rotina de compressao em duas passadas (compressao.c/compressao.h).
- Integrante 3 (CWAF-Renan): Implementacao da rotina de descompressao bit a bit com reconstrucao automatica da arvore (descompressao.c/descompressao.h) e algoritmo de verificacao de integridade por checksum CRC-32 IEEE 802.3 (checksum.c/checksum.h).
- Integrante 4 (Rodrigo Araujo - Martter): Implementacao do menu interativo no terminal (main.c), normalizacao e validacao de caminhos de arquivos, medicao de tempo com clock() e calculo de taxas, execucao da suite de testes comparativos contra o gzip, elaboracao do leia-me.txt, regras de compilacao no Makefile e redacao deste Relatorio Tecnico.

Relatorio concluido e aprovado pela equipe para submissao no Moodle - UFABC 2026.3.